

模块化与组件化开发

— 复习要点

CH09 计算机学院 | 涵盖教材 pp.2-40 (除案例全图页)

一、前端模块化概述 [p.2]

模块化分类 [p.2]

- **外部模块化** (External Modularization): 引入第三方包或插件, 通常由一个或多个 JS 文件组成。
- **内部模块化** (Internal Modularization): 工程内部的分层或分类, 内部一个模块一般由一个 JS 文件表示。

模块化的主要内容 [p.2]

- 外部模块管理: Node.js 和 NPM (包管理器)。
- 内部模块组织: CommonJS、ES6 Module 和构建工具 (Webpack 等)。
- 编译与转换: 模块源码到目标代码, 转译工具如 Babel。

二、Promise 对象 [p.6-15]

基本概念 [p.6]

- Promise 是**异步编程**的解决方案, 比传统 (事件 + 回调) 更合理强大。
- 由社区最早提出, ECMAScript 将其写入语言标准, 原生提供。
- Promise 是一个存取数据的**容器**, 封装基于 XMLHttpRequest 异步请求后的状态和值。
- 从语法上说, Promise 是一个对象, 提供**统一 API**, 各种异步操作均可用相同方法处理。

同步与异步 [p.7]

- **同步** (Synchronous): 代码顺序执行, 上一步完成后才执行下一步。
- **异步** (Asynchronous): 不等待上一步完成即继续执行。
- 核心问题: **异步代码无法通过 return 返回值**。

回调函数的问题 [p.8-9]

- 回调函数 (Callback) 是异步编程的早期方案: 将后续逻辑作为函数参数传入。
- 多次嵌套回调造成**回调地狱** (Callback Hell): 代码层层缩进, 可读性差, 调试困难, 维护成本高。

Promise 基本用法 [p.10]

- 构造函数**必须有一个函数作为参数**: `new Promise((resolve, reject) => {...})`

- 该函数的两个参数 `resolve` 和 `reject` 也是函数: 分别在**成功**和**失败**时调用进行存值。
- 通过实例的 `.then()` 方法中的两个回调分别在成功时**取值**或失败时**处理**。
- Promise 对象维护两个内部属性:
 - `PromiseResult`: 保存异步结果的值。
 - `PromiseState`: 保存状态 (`pending` / `fulfilled` / `rejected`)。
- `.then()` 监听 `PromiseState` 的变化, 选择对应的回调执行。

Promise 的三种状态 [p.11]

状态	含义	说明
<code>pending</code>	进行中	初始状态, 既未成功也未失败
<code>fulfilled</code>	已成功	操作成功完成
<code>rejected</code>	已失败	操作失败

Promise 的主要特点 [p.11]

1. 状态不受外界影响: 只有异步操作的结果可决定状态, 任何其他操作无法改变。
2. 状态一旦改变就凝固:
 - 只能从 `pending` → `fulfilled`, 或 `pending` → `rejected`。
 - 改变后不再变化, 此时称为 **resolved** (已定型)。
3. 定型后可立即取值: 与事件不同, 即使状态已定型, 再添加回调函数, 也会**立即得到结果**。

使用范例要点 [p.12-13]

- 范例一: 基本创建与使用, 演示 `resolve/reject` 和 `.then()` 的配合。
- 范例二: 结合 `setTimeout` 模拟异步场景。

链式调用 [p.14]

- `.then()` 返回**新的 Promise 对象**, 实现链式调用 (Chaining)。
- 用链式调用**替代回调函数嵌套**, 使代码扁平化, 消除回调地狱。
- 每个 `.then()` 中的返回值会作为下一个 `.then()` 的输入。

Promise 的其他 API [p.15]

- `Promise.prototype.catch()`: 专门捕获 `rejected` 状态, 等价于 `.then(null, onRejected)`。
- `Promise.prototype.finally()`: 无论 `fulfilled` 或 `rejected`, 最终都会执行的回调。
- `Promise.all(iterable)`: 接受 Promise 数组, **全部成功**则返回结果数组; **任一失败**则整体 `rejected`。

三、async / await 关键字 [p.18–21]

async 关键字 [p.18]

- 用于快速创建异步函数, 是 Promise 的**语法糖** (Syntactic Sugar)。
- 实际使用中, `async` 与 `await` 组合实现**异步向同步**的转换。
- `async` 函数始终返回一个 Promise 对象。

await 关键字 [p.19]

- `await` 调用异步函数时**阻塞当前执行流**, 等待异步函数执行结束。
- 自动从返回的 Promise 对象中**取值并返回**。
- 使用规则:
 - 只能用于 `async` 声明的异步函数中。
 - 只阻塞 `async` 函数**内部**代码, 不阻塞外部代码。
- 优点: 用**同步的方式**编写和使用异步, 代码可读性大幅提升。
- 缺点: `reject` 回调被自动忽略, 必须用额外的 `try-catch` 处理失败。 [p.19]

async/await 使用注意事项 [p.20–21]

- 将嵌套的 `.then()` 链改写为顺序 `await` 调用, 代码更符合直觉。
- 注意**并行 vs 串行**: 多个不相关异步操作应用 `Promise.all()` 并行执行, 避免不必要的串行等待。
- 需要准确理解执行顺序: `await` 后的代码等价于 `.then()` 中的回调。

四、Fetch API [p.23–25]

什么是 Fetch API [p.23]

- `fetch()` 是 `XMLHttpRequest` 的**升级版**, 用于在 JS 中发出异步 HTTP 请求。
- 归属于**ES6 标准**规范, 绝大部分现代浏览器原生支持。
- 基于**Promise 对象**实现, 极大简化异步请求代码。

fetch() 发送请求 [p.24]

- 完整语法: `fetch(url, optionObject)`
- GET 请求: 可省略 `optionObject`。
- POST 请求: 在 `optionObject` 中指定 `method`: `'POST'`、`headers`、`body` 等。

fetch() 的响应对象 [p.25]

- `fetch()` 返回一个 **Response 对象** (Promise 包装)。
- 常用属性和方法:
 - `Response.ok` — 布尔值, 2xx 为 `true`, 其他 `false`。
 - `Response.status` — 整型, 真实 HTTP 状态码。
 - `response.text()` — 返回文本字符串 (返回 Promise)。
 - `response.json()` — 返回 JSON 对象 (返回 Promise)。
 - `Response.clone()` — 底层通过 Stream 流实现, 只能**读取一次**, `clone()` 创建副本。

五、Axios 库 [p.28–29]

Axios 简介 [p.28]

- 基于 Promise 的**第三方 HTTP 库**, 可用于浏览器和 Node.js。
- 本质上是对原生 XHR 的封装, 用 Promise 实现。
- Axios 封装后提供更便捷的 API:
 - **拦截请求和响应** (Interceptors): 统一添加 token、日志、错误处理等。
 - **转换请求/响应数据** (Transform)。
 - **自动转换 JSON** 数据。

Axios 使用范例要点 [p.29]

- `axios.get(url) / axios.post(url, data)` 直接返回 Promise。
- 通过 `axios.create({baseUrl, timeout})` 创建带默认配置的实例。
- 通过拦截器统一处理请求/响应逻辑。

六、Fetch API 与 Axios 对比 [p.31–32]

Fetch API [p.31]

- 优点:
 - 原生支持, 无需额外依赖。
 - Promise 风格, 比 XHR 更简洁。
 - 现代化设计, 支持 `async/await`。
 - **流式处理** (ReadableStream): 适合大文件和渐进式加载。